

CS-334: Machine Learning

Lecture 07: Regularization

(with In-Class Exercise #6)

09/17/2026

Prof. Shengpu Tang



Announcements

- HW1 solutions posted
 - Grades soon
- Office Hours: new times
 - Thursday after class till 3:30pm
 - **Friday: 10:30am-11:30am, 1-1:45pm**
 - Location: my office in MSC W302-I

HW2 Reminder

Due Sunday 9/27 11:59pm

Topics covered

- Decision Boundaries (Lec #3)
- Perceptron (Lec #4)
- GD, SGD (Lec #5)
- Linear Regression (Lec #6)
- Regularization (Lec #7)

Extra Credit due Wed 9/30 11:59pm

- Weighted Linear Regression

Submission Instructions: The homework is due on Gradescope in **two** parts.

- **Upload PDF to HW2-Written:** Create a single high-quality PDF with your answers to the non-coding problems. Your submission may be typed or handwritten (can be scanned or from a note-taking software), but the pages must be tagged with the relevant questions appropriately on Gradescope. You do not need to include code in the PDF unless otherwise instructed.
- **Submit code to the HW2-Code:** Questions marked with ✂ on the left margin are graded by a Python autograder. Your submission must include only the following files: `perceptron.py`, `linear_regression.py`, `README.txt` (no data files please). You may submit multiple times but be sure to upload *ALL* files when you re-submit; only the latest submission will be considered. The `README.txt` file must contain a **SIGNED** honor code statement that reads as follows:

```
THIS HOMEWORK IS MY OWN WORK, WRITTEN WITHOUT COPYING FROM OTHER STUDENTS  
OR DIRECTLY FROM LARGE LANGUAGE MODELS SUCH AS CHATGPT.  
Any collaboration or external resources have been properly acknowledged.  
<add details>  
/* Your_Name_Here */
```

Note about runtime: a correct implementation should take <1 minute for all experiments in this homework.

Recap: Classification

Given **training data** containing a set of **labeled examples**,
come up with a **rule** that maps an **unlabeled example** to the **correct label**.

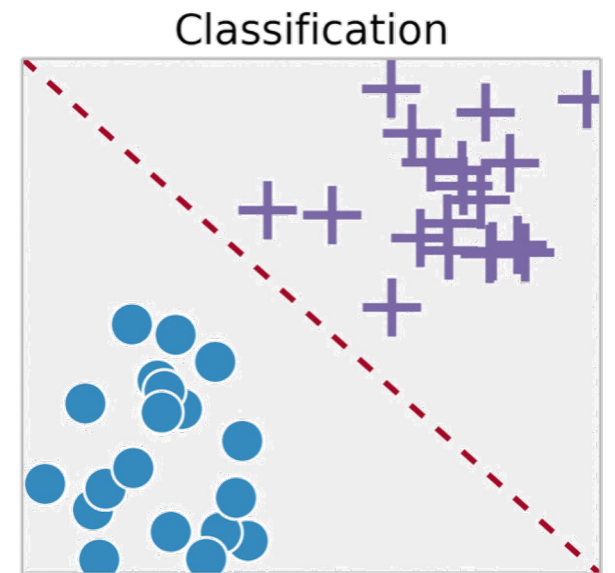
Given $D = \{\vec{x}^{(i)}, y^{(i)}\}_{i=1}^N$

$$\vec{x}^{(i)} \in \mathbb{R}^d$$

$$y^{(i)} \in \{-1, +1\}$$

Find a “good” $h : \mathbb{R}^d \rightarrow \{-1, +1\}$

Linear classification $h(\vec{x}; \vec{\theta}) = \text{sign}(\vec{\theta} \cdot \vec{x})$



Recap: Regression

Given **training data** containing a set of **labeled examples**,
come up with a **rule** that maps an **unlabeled example** to the **correct label**.

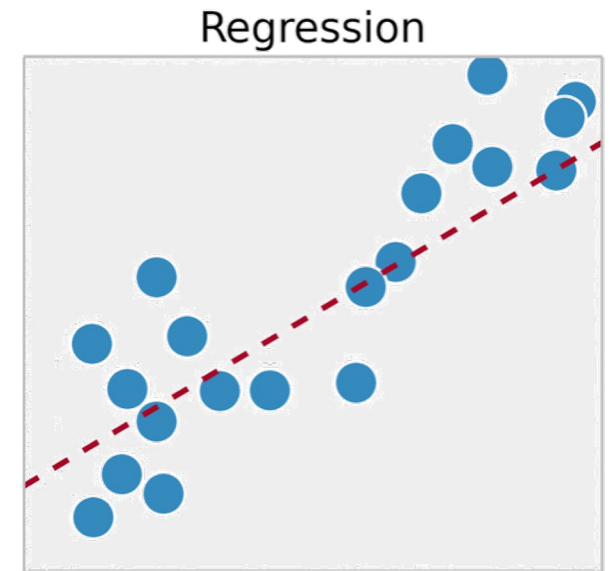
Given $D = \{\vec{x}^{(i)}, y^{(i)}\}_{i=1}^N$

$$\vec{x}^{(i)} \in \mathbb{R}^d$$

$$y^{(i)} \in \mathbb{R}$$

Find a “good” $f : \mathbb{R}^d \rightarrow \mathbb{R}$

Linear regression $f(\vec{x}; \vec{\theta}, b) = \vec{\theta} \cdot \vec{x} + b$



Recap: Linear Binary Classification

Objective: minimize empirical risk with {zero-one, perceptron, hinge} loss

$$\min_{\vec{\theta}} \frac{1}{N} \sum_{i=1}^N \text{loss}(y^{(i)}, \vec{\theta} \cdot \vec{x}^{(i)})$$

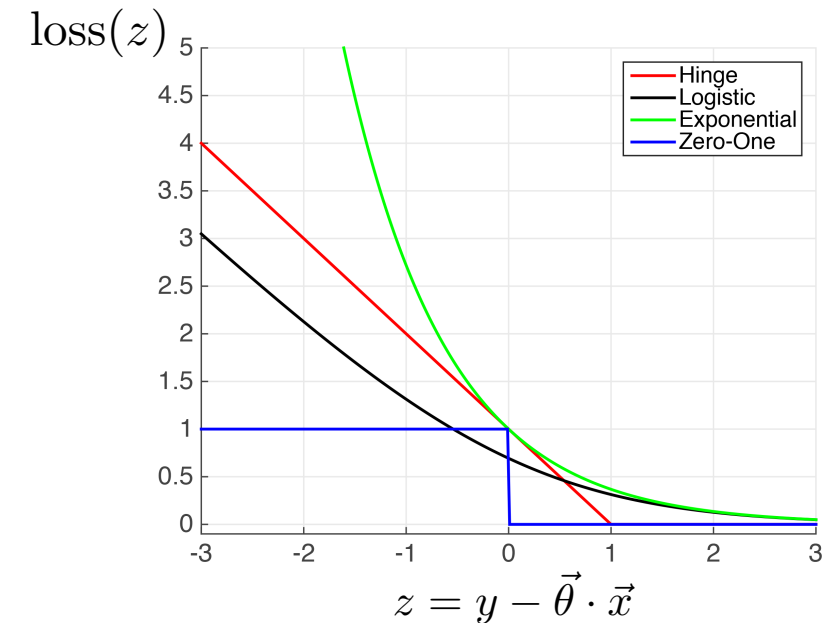
1) “Eyeballing” boundary to minimize training error

2) Perceptron algorithm (DO NOT USE IRL)

$$\vec{\theta}^{(k+1)} = \vec{\theta}^{(k)} + y^{(i)} \vec{x}^{(i)}$$

3) (Stochastic) Gradient Descent

$$\vec{\theta}^{(k+1)} = \vec{\theta}^{(k)} - \eta_k \nabla_{\vec{\theta}} \text{loss}(y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)})|_{\vec{\theta}=\vec{\theta}^{(k)}}$$



Recap: Linear Regression

Objective: minimize empirical risk with squared error loss

$$\min_{\vec{\theta}} \frac{1}{N} \sum_{i=1}^N \frac{(y^{(i)} - \vec{\theta} \cdot \vec{x}^{(i)})^2}{2}$$

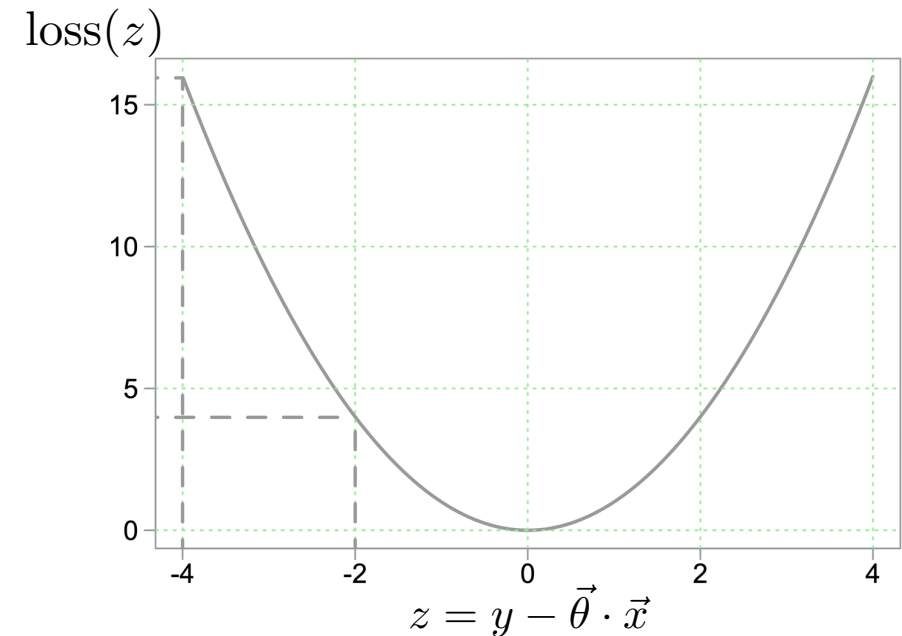
Two solutions

1) Stochastic Gradient Descent

$$\vec{\theta}^{(k+1)} = \vec{\theta}^{(k)} + \eta_k (y^{(i)} - \vec{\theta}^{(k)} \cdot \vec{x}^{(i)}) \vec{x}^{(i)}$$

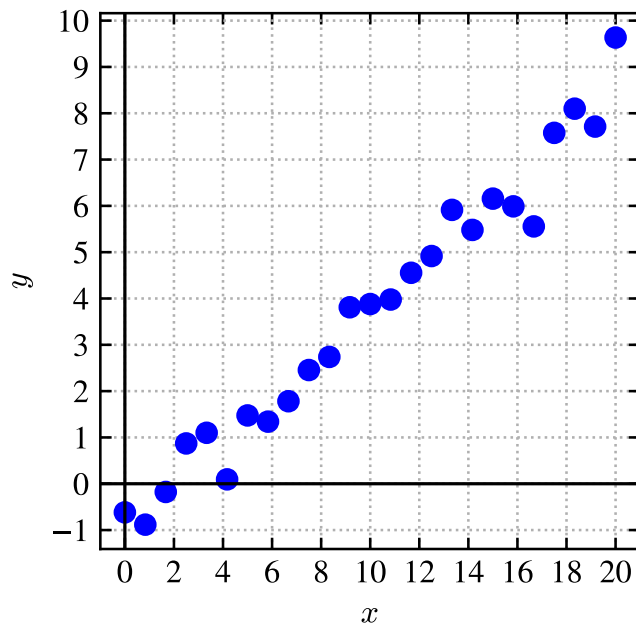
2) Closed-Form

$$\vec{\theta}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} = \mathbf{X}^+ \mathbf{y}$$

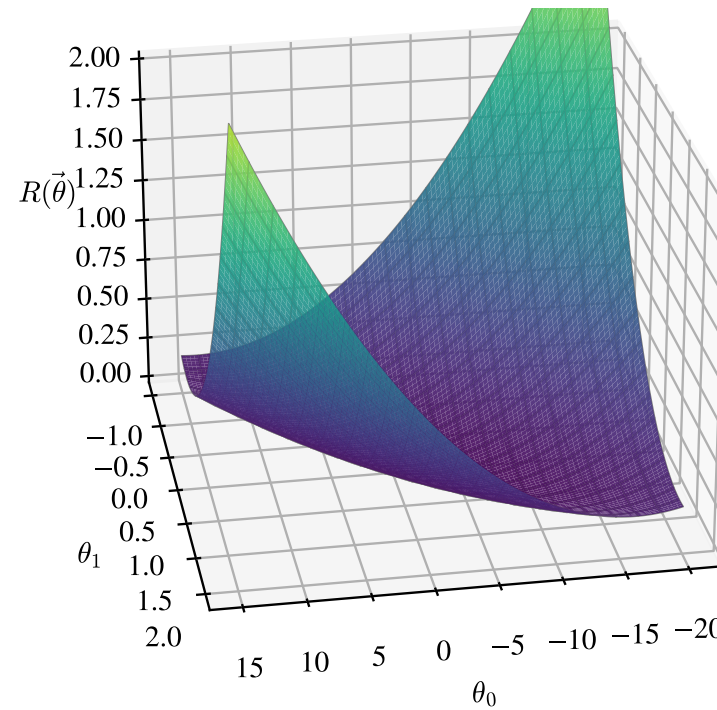


Regression Loss Surface & Contour Plot

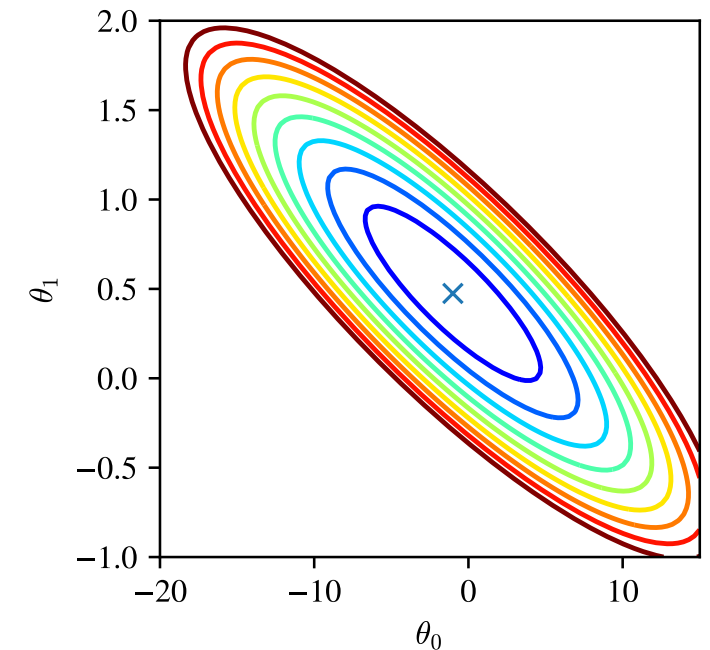
$$f(x; \theta_0, \theta_1) = \theta_0 + \theta_1 x$$



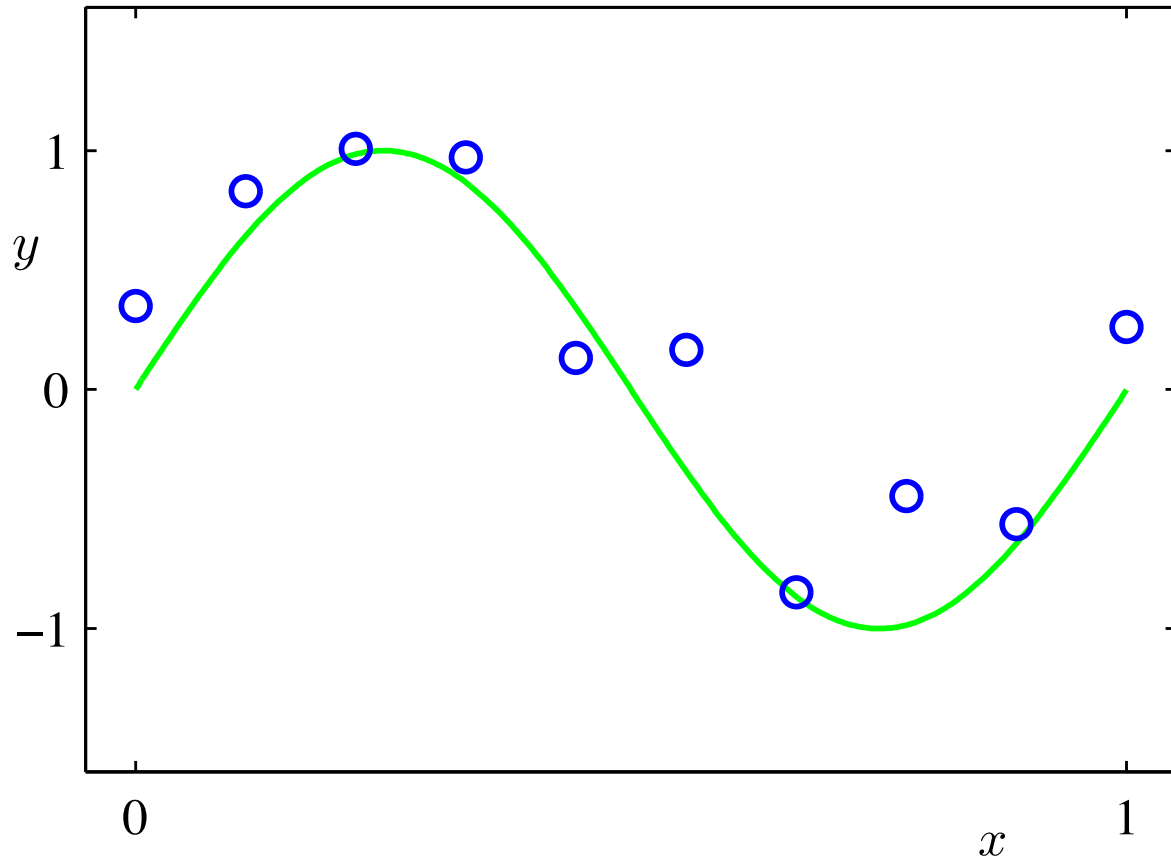
loss surface



loss contour



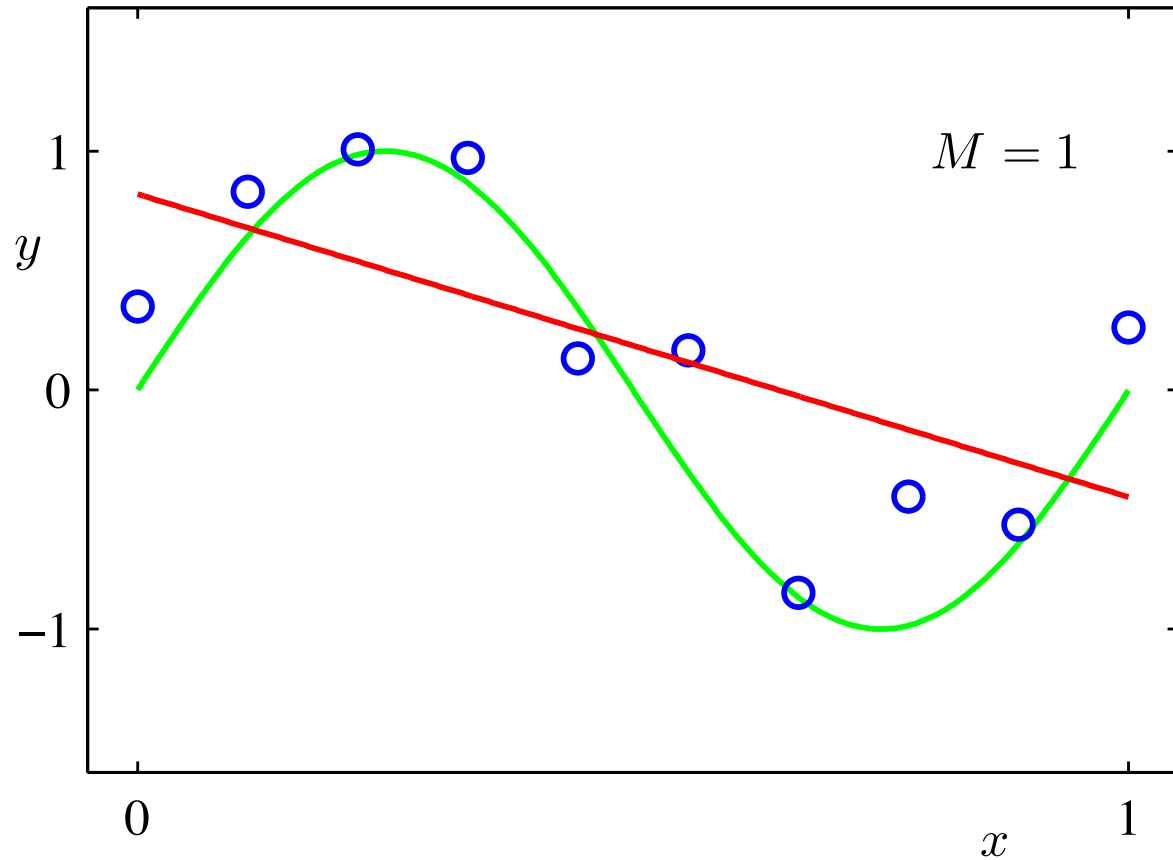
Linear Regression



How can we use linear regression to fit non-linear functions (in two dimensions)?

Image source: Christopher Bishop. *Pattern Recognition and Machine Learning*.

Linear Regression



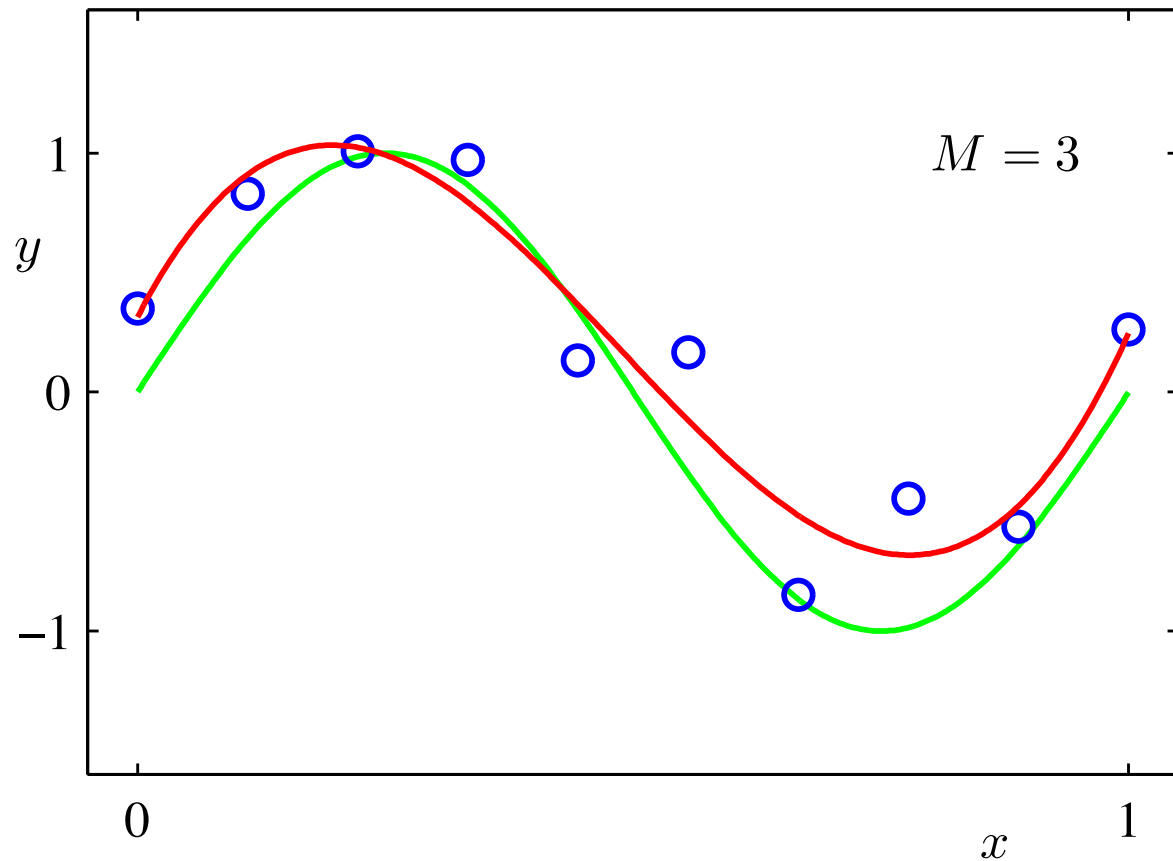
How can we use linear regression to fit non-linear functions (in two dimensions)?

$$\phi(x) = [1, x]$$

$$\vec{\theta} = [0.82, -1.27]$$

This doesn't even fit the training data well...

Polynomial Regression



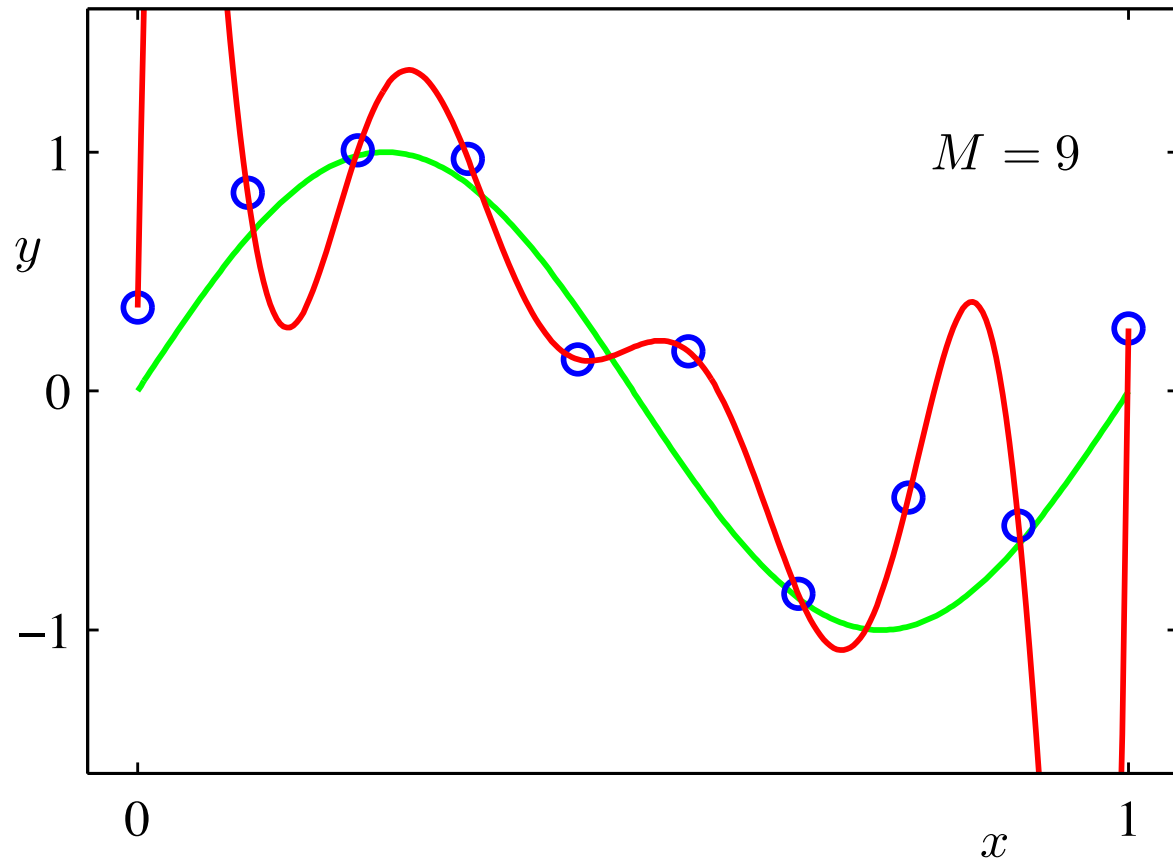
map data to higher dimension through
an explicit feature mapping

$$\phi(x) = [1, x, x^2, x^3]$$

$$\vec{\theta} = [0.31, 7.99, -25.43, 17.37]$$

model is linear in higher-dimensional space
but nonlinear in original space

Polynomial Regression



$$\phi(x) = [1, x, x^2, \dots, x^9]$$

$$\vec{\theta} = \begin{bmatrix} 0.35 \\ 232.37 \\ -5321.83 \\ 48568.31 \\ -231639.30 \\ 640042.26 \\ -1061800.52 \\ 1042400.18 \\ -557682.99 \\ 125201.43 \end{bmatrix}$$

This fits the training data (very) well so what's wrong?

Generalization (recall Lectures #1 & #3)

What we ultimately want is a model that works well on **unseen data**.

But... we've been talking about **minimizing empirical risk on training data**.

$$D = \{\vec{x}^{(i)}, y^{(i)}\}_{i=1}^N$$

$$R(\vec{\theta}; D) = \frac{1}{N} \sum_{i=1}^N \text{loss}(\vec{x}^{(i)}, y^{(i)}, \vec{\theta})$$

$$D_{\text{test}} = \{\vec{x}^{(i)}, y^{(i)}\}_{i=1}^{N'}$$

$$R(\vec{\theta}; D_{\text{test}}) = \frac{1}{N'} \sum_{i=1}^{N'} \text{loss}(\vec{x}^{(i)}, y^{(i)}, \vec{\theta})$$

Generalization Gap $R(\vec{\theta}; D_{\text{test}}) - R(\vec{\theta}; D)$

Generalization

What we ultimately want is a model that works well on **unseen data**.

But... we've been talking about **minimizing empirical risk on training data**.

If we optimize $\vec{\theta}$ by minimizing $R(\vec{\theta}; D)$

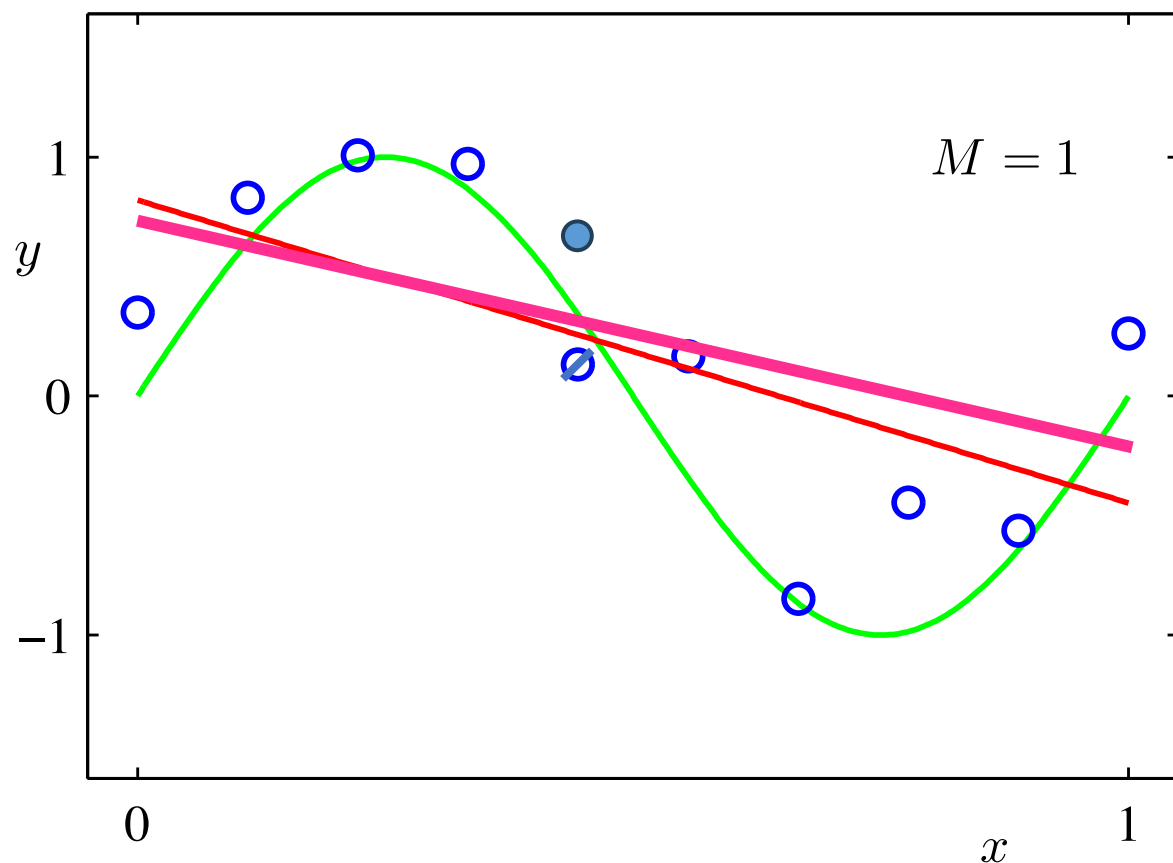
Then $R(\vec{\theta}; D_{\text{test}})$ may be large for two reasons

Underfitting
“Bias”
“Approximation Error”

&

Overfitting
“Variance”
“Estimation Error”

Underfitting / Bias / Approximation Error



Imagine training the model on multiple datasets

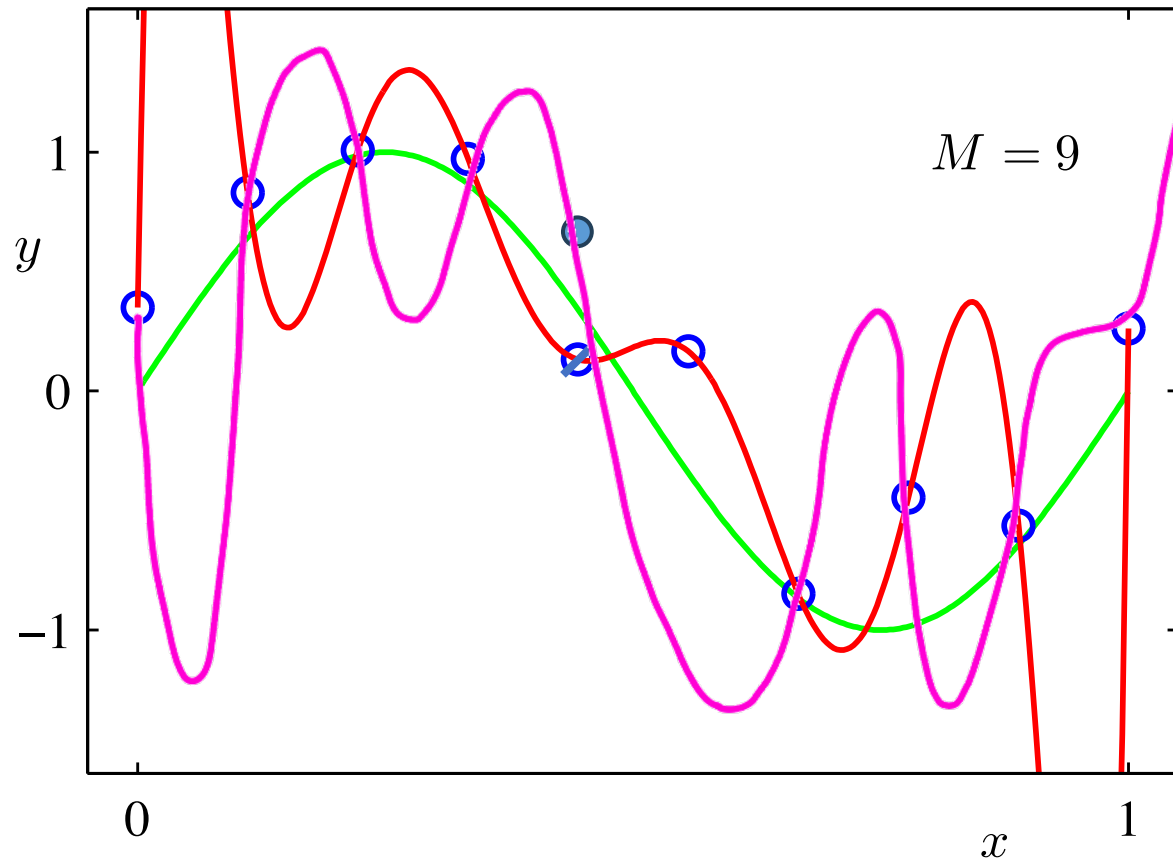
On average, is the prediction of our model close to the true value?

“Error of approximating real world with a model”

- Poor on training set
- Poor on test set

as $N \uparrow$, doesn't really help

Overfitting / Variance / Estimation Error



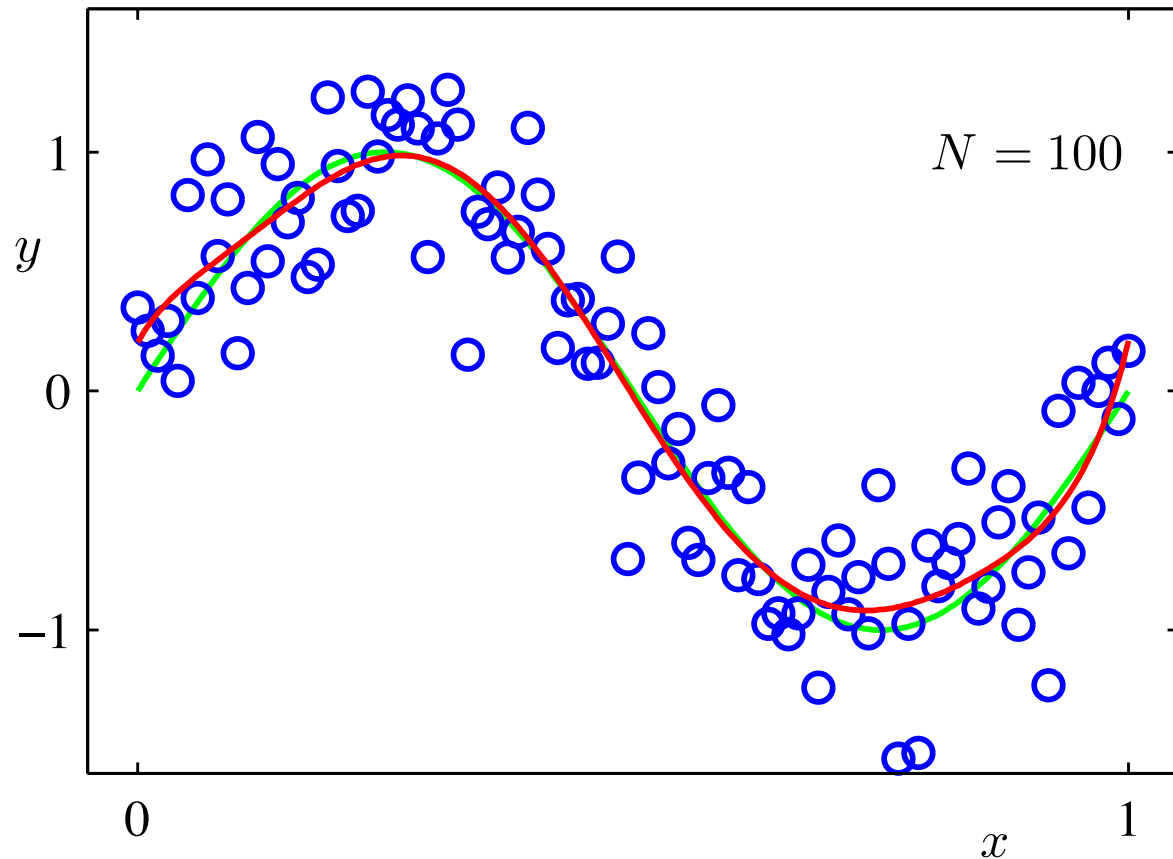
Imagine training the model on multiple datasets

How much do the predictions of our model trained on different datasets vary (around their average)?

“Are we picking up on the signal or the noise?”

- (Very) good on training set
- Poor on test set

Overfitting / Variance / Estimation Error



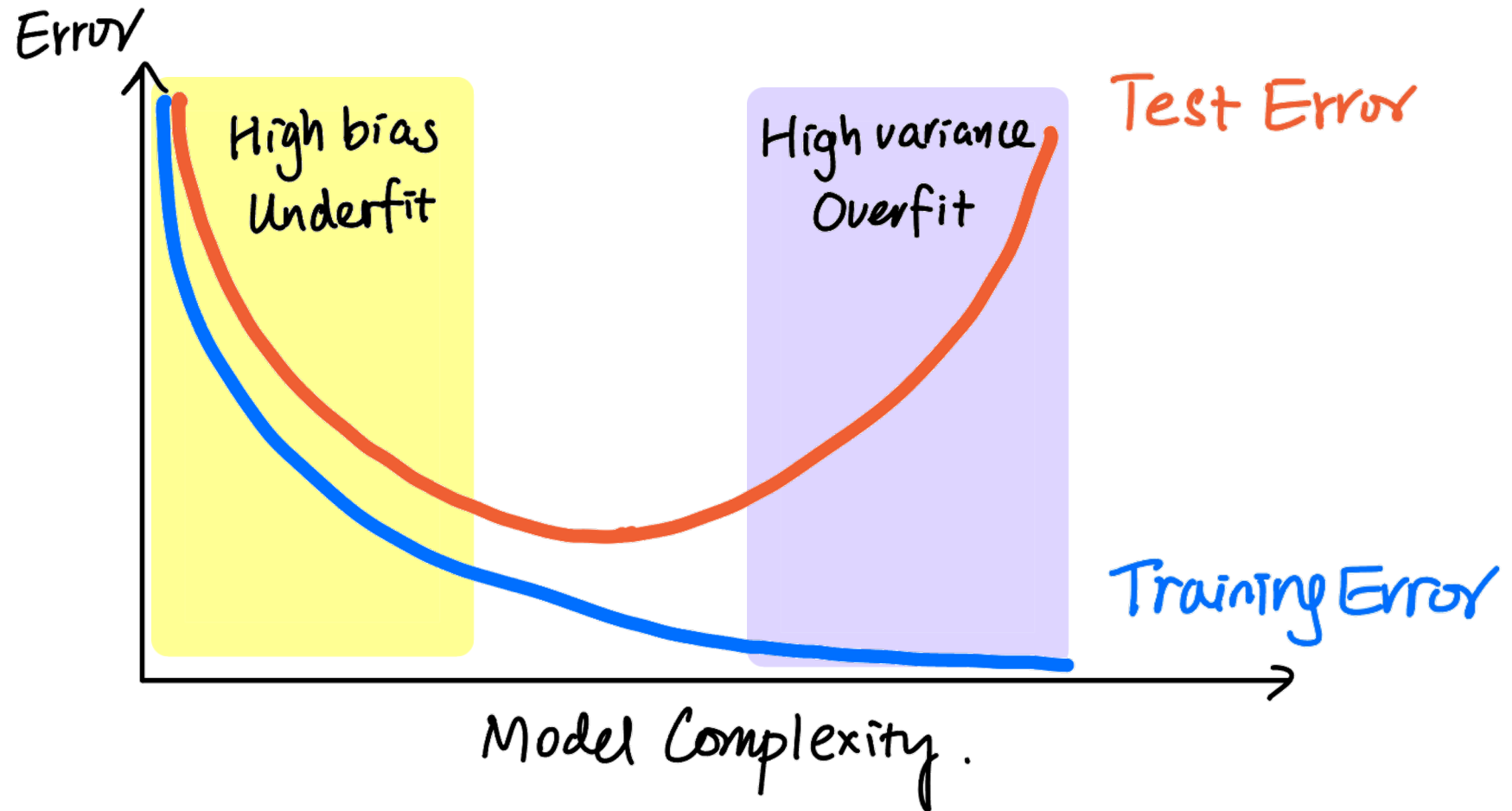
Imagine training the model on multiple datasets

How much do the predictions of our model trained on different datasets vary around their average?

- (Very) good on training set
- Poor on test set

as $N \uparrow$, variance $\downarrow$

Bias-Variance Trade-Off



How to Control Model Complexity?

One solution: Vary degree of polynomial features.

Another solution: Regularization.

See notes.

Lecture 7. Regularization

Regularization: a "knob" for controlling model complexity.

$$J(\vec{\theta}) = R_N(\vec{\theta}) + \underbrace{\lambda \Omega(\vec{\theta})}_{\substack{\text{regularizer / regularization penalty.} \\ \text{regularization term} \\ \text{regularization strength — balances how well we fit the data} \\ \lambda > 0. \\ \text{vs. complexity of model}}}$$

Common regularizers:

$$L_2: \Omega(\vec{\theta}) = \frac{\|\vec{\theta}\|_2^2}{2} = \frac{1}{2} \sum_{j=1}^d \theta_j^2 \quad \text{"Ridge"}$$

$$L_1: \Omega(\vec{\theta}) = \|\vec{\theta}\|_1 = \sum_{j=1}^d |\theta_j| \quad \text{"LASSO" — leads to sparse solutions.}$$

$$\text{Elastic net: } \Omega(\vec{\theta}) = \lambda_2 \|\vec{\theta}\|_2^2 + \lambda_1 \|\vec{\theta}\|_1$$

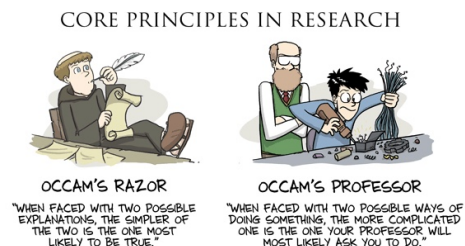
* Explore more in HW3

Effect of Regularization

- When minimizing $J(\vec{\theta})$, we're still trying to minimize $R_N(\vec{\theta})$ but at the same time minimize $\Omega(\vec{\theta})$
 - push model parameters toward smaller values
 - resist setting parameters away from default of zero unless data strongly suggests otherwise
- Why small parameter values in $\vec{\theta}$ are good?
 - limited effect of small perturbations of input features on model output.
 - if $\theta_j = 0$ exactly, then that j -th feature is effectively unused.

William of Occam
Occam's Razor (13th century philosopher)

"When you have two competing hypotheses, the simpler one is preferred"



algorithms/
How does regularization affect our solutions for regression?

Ridge Regression. (L2-regularized linear regression)

$$J(\vec{\theta}) = \sum_{i=1}^N \frac{(y^{(i)} - \vec{\theta} \cdot \vec{x}^{(i)})^2}{2} + \underbrace{\lambda \frac{\|\vec{\theta}\|^2}{2}}_{\text{extra term here}}$$

small constant here:
 factor of $\frac{1}{N}$ "absorbed" into λ .

► SGD

Let's calculate gradient:

$$\nabla_{\vec{\theta}} \left(\frac{(y^{(i)} - \vec{\theta} \cdot \vec{x}^{(i)})^2}{2} + \lambda \frac{\|\vec{\theta}\|^2}{2} \right)$$

Gradient of squared L2-norm.

$$\|\vec{\theta}\|^2 = \theta_1^2 + \dots + \theta_d^2$$

$$\frac{\partial}{\partial \theta_j} (\|\vec{\theta}\|^2) = 2\theta_j$$

$$\begin{aligned} \nabla_{\vec{\theta}} (\vec{\theta} \cdot \vec{\theta}) &= \left[\frac{\partial}{\partial \theta_1} (\|\vec{\theta}\|^2), \dots, \frac{\partial}{\partial \theta_d} (\|\vec{\theta}\|^2) \right]^T \\ &= [2\theta_1, 2\theta_2, \dots, 2\theta_d]^T = 2\vec{\theta}. \end{aligned}$$

update rule:

$$\begin{aligned} \vec{\theta}^{(k+1)} &= \vec{\theta}^{(k)} - \eta_k \left[(y^{(i)} - \vec{\theta}^{(k)} \cdot \vec{x}^{(i)}) (-\vec{x}^{(i)}) + \lambda \vec{\theta}^{(k)} \right] \\ &= \underbrace{(1 - \eta_k \lambda)}_{\text{shrink parameters toward zero in each update}} \vec{\theta}^{(k)} + \underbrace{\eta_k (y^{(i)} - \vec{\theta}^{(k)} \cdot \vec{x}^{(i)}) \vec{x}^{(i)}}_{\text{same as before}} \end{aligned}$$

► Closed-form

$$J(\vec{\theta}) = \frac{1}{2} (\mathbf{X}\vec{\theta} - \mathbf{y})^T (\mathbf{X}\vec{\theta} - \mathbf{y}) + \frac{\lambda}{2} \vec{\theta}^T \vec{\theta}$$

$$\begin{aligned} \nabla_{\vec{\theta}} J(\vec{\theta}) &= (\mathbf{X}^T \mathbf{X}) \vec{\theta} - \mathbf{X}^T \mathbf{y} + \lambda \vec{\theta} \\ &= (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}_d) \vec{\theta} - \mathbf{X}^T \mathbf{y} \end{aligned}$$

$d \times d$ identity matrix

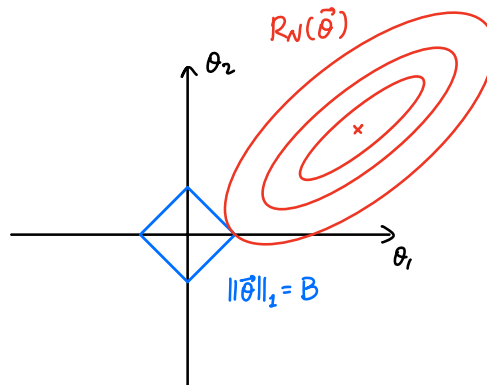
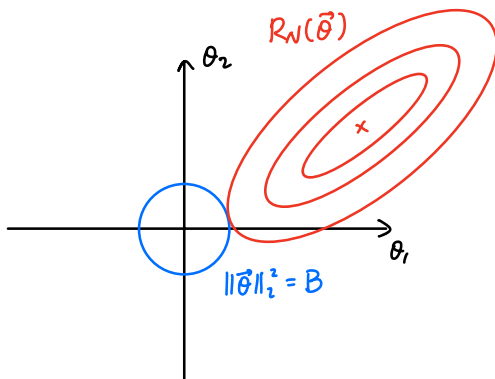
$$\nabla_{\vec{\theta}} J(\vec{\theta})|_{\vec{\theta}=\vec{\theta}^*} = 0 \quad \Rightarrow \quad \vec{\theta}^* = \underbrace{(\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}_d)^{-1}}_{\text{always invertible.}} \mathbf{X}^T \mathbf{y}$$

Geometric Interpretation of Regularization.

$$\begin{array}{ccc} \min_{\vec{\theta}} R_N(\vec{\theta}) + \lambda \Omega(\vec{\theta}) & \xLeftrightarrow{\text{equivalent}} & \min_{\vec{\theta}} R_N(\vec{\theta}) \text{ s.t. } \Omega(\vec{\theta}) \leq B \\ \text{unconstrained optimization} & \text{by Lagrange multiplier theorem} & \text{constrained optimization} \end{array}$$

Where is the optimal solution of the constrained optimization problem?

- Method of Lagrange multiplier:
the point where level curve is tangent with the constraint is the minimum.
"just touching"
- Intuition: two forces at play
 $\Omega(\vec{\theta}) < B$ shrinks $\vec{\theta}$ to be within the circle/square } settle at a point
 $\min R_N(\vec{\theta})$ moves $\vec{\theta}$ towards the unconstrained optimum } where the two curves
are tangent.



→ In-Class Exercise #6